Simulador de Planta con Python y Snap7 para TIA Portal

https://gemini.google.com/app/c9a022ab913ae5ec

User prompt: Hola Gemini. El día de hoy me gustaría poder probar el funcionamiento de mi programación del proyecto capstone. Sin embargo, siento que usando PLCSIM tradicional no puedo ver el proceso real funcionando en la planta. Por esta razón, quiero desarrollar mi propio simulador de la planta real usando la librería de Python para snap7. Actualmente en mi computadora tengo instalados: - TIA Portal V20 - WINCC para el HMI - SIMATIC S7-PLCSIM Vi en internet que necesito PLCSIM Advanced, ¿no habrá alguna manera de comunicar el PLC con el proceso sin esto? Me gustaría que me des una guía en forma de una serie de pasos.

Response: ¡Hola! Claro que sí, estás de suerte: **no necesitas obligatoriamente PLCSIM Advanced** para comunicar tu PLC virtual con un script de Python usando Snap7.

Tal como ocurre cuando se utiliza MATLAB como cliente Snap7 para leer y escribir bloques de datos (como el DB de IoT), puedes lograr exactamente lo mismo con **Python** y la librería `python-snap7`. El S7-PLCSIM estándar (para S7-1200 y S7-1500) habilita una tarjeta de red virtual y una dirección IP (por ejemplo, `192.168.0.1`) que permite comunicaciones ISO-on-TCP estándar, permitiendo que Python se conecte sin problemas.

A continuación, te presento una guía paso a paso para configurar tu simulador de planta en Python:

Paso 1: Configurar el Bloque de Datos (DB) como "No Optimizado" en TIA Portal

Para que Snap7 pueda leer y escribir variables mediante direcciones físicas exactas (*offsets* de byte), el bloque de datos no debe usar el acceso optimizado de Siemens.

1. En tu proyecto de TIA Portal V20, crea un Bloque de Datos (por ejemplo, el `DB200` o el que prefieras usar para el intercambio).
2. Haz clic derecho sobre el DB, selecciona **Propiedades** (*Properties*).
3. En la sección *General*, desmarca la casilla **"Optimized block access"** (Acceso optimizado al bloque).
4. Define dentro de este DB las variables de tu planta (por ejemplo: entradas de sensores, salidas de motores, temperaturas, contadores). Al guardarlo, TIA Portal te asignará un *offset* en bytes a cada variable (ej. `DBW0`, `DBB2`, etc.).

Paso 2: Iniciar la simulación en S7-PLCSIM

1. Selecciona tu PLC en el árbol de proyectos de TIA Portal y haz clic en el botón de **Simular** (*Start simulation*).
2. Se abrirá la interfaz de **S7-PLCSIM**. Configura la interfaz PG/PC correctamente (asegúrate de que esté apuntando a la red virtual de PLCSIM, como `PN/IE` o `PLCSIM`).
3. Realiza la carga (*Download*) del proyecto hacia el PLC virtual y asegúrate de poner el PLC en modo **RUN**.
4. Anota la dirección IP que tiene asignada el PLC virtual en la configuración de hardware (usualmente `192.168.0.1`).

Paso 3: Instalar la librería `python-snap7` en tu entorno

1. Abre tu terminal o consola de comandos (CMD, PowerShell o terminal de VS Code).
2. Instala la librería ejecutando el comando:

   ```
   pip install python-snap7
   ```

   (Nota: Asegúrate de tener Python instalado en tu equipo).

Paso 4: Desarrollar el script de Python (Simulador de Planta)

Crea un archivo de Python (por ejemplo, `simulador_planta.py`). Aquí tienes una estructura base para conectarte al PLC, leer las salidas de control y escribir las señales de los sensores simulados:

```
import time
import snap7
from snap7.util import *
from snap7.types import *

# Configuración de conexión
PLC_IP = "192.168.0.1"  # IP de tu PLCSIM
RACK = 0
SLOT = 1                # 1 para S7-1200, 2 para S7-1500 (ajusta según tu CPU)
DB_NUMBER = 200         # El número de tu DB no optimizado[cite: 1]

# Inicializar cliente Snap7
client = snap7.client.Client()
client.connect(PLC_IP, RACK, SLOT)

if client.get_connected():
    print("¡Conexión exitosa con S7-PLCSIM!")
else:
    print("Error al conectar con el PLC.")
    exit()

try:
    while True:
        # 1. Leer los datos enviados por el PLC (ej. Actuadores Q o variables del DB)
        # Por ejemplo, leemos 10 bytes del DB200
        db_data = client.db_read(DB_NUMBER, 0, 10)

        # Ejemplo: extraer una variable booleana o numérica del byte offset
        # motor_encendido = get_bool(db_data, byte_index=0, bool_index=0)

        # 2. Lógica del proceso físico de la planta en Python
        # Aquí simulas la física: si el motor está encendido, la temperatura sube, etc.

        # 3. Escribir de vuelta las variables de sensores hacia el PLC (para que el PLC las lea)
        # set_bool(db_data, byte_index=0, bool_index=1, val=True)
        # client.db_write(DB_NUMBER, 0, db_data)

        time.sleep(0.1)  # Ciclo de actualización de 100ms

except KeyboardInterrupt:
    print("\nSimulación detenida por el usuario.")
    client.disconnect()
```

Paso 5: Integración con WinCC (HMI)

Lo genial de este enfoque es que **WinCC** también puede conectarse al mismo S7-PLCSIM en segundo plano. Mientras Python lee y escribe en los bloques de datos para simular el comportamiento físico de la planta, tu HMI en WinCC reflejará esos cambios en tiempo real, permitiéndote operar la planta virtual de manera idéntica a un entorno industrial real.

¿Qué tipo de proceso o planta específica estás desarrollando en tu proyecto capstone para afinar los detalles de las variables del script?

User prompt: Fíjate que en TIA Portal V20 no aparece la casilla de Optimized Block Access, creo que ya había tenido ese problema anteriromente. ¿Qué hago? Te dejo adjunta una imagen del DB creado

Response: En TIA Portal, la opción no está en la pestaña principal de "General", sino en el submenú de atributos específicos del bloque.

1 **Navegar a la sección de Atributos**

Propiedades del DB

En la ventana de propiedades que tienes abierta actualmente, mira el menú de la izquierda y haz clic en **Attributes** (*Atributos*), que se encuentra justo debajo de *Protection*. *Cómo verificar:* La sección central de la ventana cambiará para mostrar las opciones avanzadas y de memoria del bloque de datos.

2 **Desmarcar el acceso optimizado**

Configuración de accesibilidad

Busca la casilla llamada **Optimized block access** (*Acceso optimizado al bloque*) y desmárcala. Haz clic en el botón **OK**. *Cómo verificar:* Al volver a la tabla de variables de tu DB, notarás que ahora aparece la columna **Offset** con las direcciones físicas en bytes (por ejemplo, `0.0`, `2.0`) necesarias para Snap7.

3 **Compilar el bloque de datos**

Actualización del sistema

Haz clic derecho sobre tu `DB_Simulator` en el árbol de proyectos y selecciona **Compile > Software (rebuild all)**. *Cómo verificar:* La consola inferior mostrará que el bloque se ha compilado correctamente sin errores, aplicando el direccionamiento absoluto.

User prompt: Parece que la casilla no se puede marcar. Repliqué todas las entradas y salidas del PLC en el nuevo DB y sí se colocaron las direcciones físicas en bytes necesarias para Snap7 después de compilar.

Response: Es totalmente normal que aparezca desmarcada y en gris. Si la casilla de **Optimized block access** está desmarcada y no se puede modificar, significa que **ya está desactivada por defecto**. El hecho de que la columna **Offset** aparezca con valores como `0.0`, `0.1`, `2.0` confirma que tu bloque de datos ya está configurado en modo no optimizado (direccionamiento absoluto), que es justo lo que requiere Snap7.

1 **Verificar el direccionamiento absoluto**

Paso 1

Revisa que todas tus variables tengan asignada una dirección en la columna **Offset** (por ejemplo, `Int` en `2.0`, `Bool` en `4.0`). *Cómo verificar:* Si las direcciones numéricas de byte y bit son visibles en cada fila, el bloque está listo para comunicación externa.

2 Compilar y descargar al PLC

Paso 2

Haz clic derecho sobre tu bloque de datos (`DB_Simulator`), selecciona **Compile > Software (rebuild all)** y luego carga los cambios al S7-PLCSIM. *Cómo verificar:* La consola debe mostrar la compilación exitosa sin errores y el PLC virtual debe aceptar la actualización en línea.

3 Ejecutar el script de prueba en Python

Paso 3

Usa la librería `python-snap7` para leer los datos del `DB300` utilizando el número de bloque y el offset correspondiente. *Cómo verificar:* El script debe imprimir la lectura exitosa de los bytes sin arrojar errores de conexión o de direccionamiento.